

Customer Sync Pipeline

Code:

```
import os
import logging
from datetime import datetime, timedelta

import pandas as pd

# from airflow.decorators import dag
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
from airflow.exceptions import AirflowException

logger = logging.getLogger(__name__)

def alert_on_failure(context):
    ti = context["task_instance"]
    dag_id = ti.dag_id
    task_id = ti.task_id
    print(f"ALERT: Task {task_id} in DAG {dag_id} has failed. Check the logs.")

def validate_file():
    try:
        file_path = "data/customers_data.csv"

        if not os.path.exists(file_path):
            raise AirflowException(
                "customers_data.csv not found"
            )

        logger.info(
            "CSV file found – proceeding with load."
        )

        return file_path

    except AirflowException:
        raise

    except Exception as e:
```

```

        logger.error(f"Error in validate_file: {str(e)}")
        raise AirflowException(f"validate_file failed: {str(e)}")

def create_and_load():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create table
        create_table_sql = """
        CREATE TABLE IF NOT EXISTS RAW_CUSTOMERS (
            customer_id INT,
            name VARCHAR,
            email VARCHAR,
            city VARCHAR,
            signup_date DATE,
            plan VARCHAR,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_table_sql)

        logger.info("RAW_CUSTOMERS table ready.")

        # Idempotency delete
        hook.run("""
            DELETE FROM RAW_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        logger.info("Cleared today's existing RAW_CUSTOMERS rows.")

        # Read CSV and insert rows
        # Read CSV and insert rows
        file_path = "data/customers_data.csv"
        df = pd.read_csv(file_path)

        values_list = []

        for _, row in df.iterrows():

            name = str(row['name']).replace("'", "'")
            email = str(row['email']).replace("'", "'")
            city = str(row['city']).replace("'", "'")

```

```

plan = str(row['plan']).replace("'", '"')

values_list.append(f"""
    (
        {row['customer_id']},
        '{name}',
        '{email}',
        '{city}',
        '{row['signup_date']}',
        '{plan}',
        CURRENT_TIMESTAMP()
    )
""")

# Insert in batches of 100
batch_size = 100

for i in range(0, len(values_list), batch_size):

    batch = values_list[i: i + batch_size]

    insert_sql = f"""
        INSERT INTO RAW_CUSTOMERS (
            customer_id,
            name,
            email,
            city,
            signup_date,
            plan,
            loaded_at
        )
        VALUES {'', '.join(batch)};
    """

    hook.run(insert_sql)

    logger.info(
        f"Inserted batch {i // batch_size + 1} "
        f"({len(batch)} rows)."
    )

    logger.info(f"Loaded {len(df)} rows into RAW_CUSTOMERS.")

except AirflowException:
    raise

except Exception as e:
    logger.error(f"Error in create_and_load: {str(e)}")

```

```

        raise AirflowException(f"create_and_load failed: {str(e)}")

def enrich_customers():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create enriched table
        create_enriched_sql = """
        CREATE TABLE IF NOT EXISTS ENRICHED_CUSTOMERS (
            customer_id INT,
            name VARCHAR,
            email VARCHAR,
            city VARCHAR,
            signup_date DATE,
            plan VARCHAR,
            customer_tier VARCHAR,
            days_since_signup INT,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_enriched_sql)

        logger.info("ENRICHED_CUSTOMERS table ready.")

        # Idempotency delete
        hook.run("""
            DELETE FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        logger.info("Cleared today's existing ENRICHED_CUSTOMERS rows.")

        # Insert enriched data
        insert_enriched_sql = """
        INSERT INTO ENRICHED_CUSTOMERS (
            customer_id,
            name,
            email,
            city,
            signup_date,
            plan,
            customer_tier,
            days_since_signup,

```

```

        loaded_at
    )
    SELECT
        customer_id,
        name,
        email,
        city,
        signup_date,
        plan,
        CASE WHEN plan = 'premium' THEN 'Premium Tier' ELSE 'Basic Tier'
END,
        DATEDIFF('day', signup_date, CURRENT_DATE()),
        CURRENT_TIMESTAMP()
    FROM RAW_CUSTOMERS
    WHERE loaded_at::DATE = CURRENT_DATE();
    """

    hook.run(insert_enriched_sql)

    # Log enriched row count
    count = hook.get_first("""
        SELECT COUNT(*)
        FROM ENRICHED_CUSTOMERS
        WHERE loaded_at::DATE = CURRENT_DATE();
    """)[0]

    logger.info(f"{count} enriched rows created in ENRICHED_CUSTOMERS.")

except AirflowException:
    raise

except Exception as e:
    logger.error(f"Error in enrich_customers: {str(e)}")
    raise AirflowException(f"enrich_customers failed: {str(e)}")

def quality_check():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Check 1 – NULL check
        null_count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE (name IS NULL OR email IS NULL)

```

```

        AND loaded_at::DATE = CURRENT_DATE();
    """)[0]

    logger.info(f"NULL check result: {null_count} rows with nulls.")

    # Check 2 – Duplicate check
    duplicate_count = hook.get_first("""
        SELECT COUNT(*) - COUNT(DISTINCT customer_id)
        FROM ENRICHED_CUSTOMERS
        WHERE loaded_at::DATE = CURRENT_DATE();
    """)[0]

    logger.info(f"Duplicate check result: {duplicate_count} duplicate
customer_ids.")

    if null_count == 0 and duplicate_count == 0:
        logger.info("Both checks passed – routing to all_clear.")
        return "all_clear"
    else:
        logger.warning("One or more checks failed – routing to
data_warning.")
        return "data_warning"

except AirflowException:
    raise

except Exception as e:
    logger.error(f"Error in quality_check: {str(e)}")
    raise AirflowException(f"quality_check failed: {str(e)}")

def all_clear():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        logger.info(
            "Quality checks passed – customer data is clean."
        )

        count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]

```

```

        logger.info(f"Total enriched rows for today: {count}")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in all_clear: {str(e)}")
        raise AirflowException(f"all_clear failed: {str(e)}")

def data_warning():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        null_count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE (name IS NULL OR email IS NULL)
            AND loaded_at::DATE = CURRENT_DATE();
        """)[0]

        duplicate_count = hook.get_first("""
            SELECT COUNT(*) - COUNT(DISTINCT customer_id)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]

        logger.warning(
            "WARNING: Data quality issues detected in customer load."
        )

        if null_count > 0:
            logger.warning(f"NULL check FAILED – {null_count} rows with null
name or email.")

        if duplicate_count > 0:
            logger.warning(f"Duplicate check FAILED – {duplicate_count}
duplicate customer_ids.")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in data_warning: {str(e)}")
        raise AirflowException(f"data_warning failed: {str(e)}")

```

```

def final_summary():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        summary = hook.get_records("""
            SELECT
                COUNT(*) AS total_customers,
                SUM(CASE WHEN customer_tier = 'Premium Tier' THEN 1 ELSE 0
END) AS premium_count,
                SUM(CASE WHEN customer_tier = 'Basic Tier' THEN 1 ELSE 0 END)
AS basic_count,
                ROUND(AVG(days_since_signup), 1) AS avg_days_since_signup,
                MODE(city) AS top_city
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        row = summary[0]

        logger.info(f"Total Customers: {row[0]}")
        logger.info(f"Premium Tier: {row[1]}")
        logger.info(f"Basic Tier: {row[2]}")
        logger.info(f"Average Days Since Signup: {row[3]}")
        logger.info(f"City with Most Customers: {row[4]}")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in final_summary: {str(e)}")
        raise AirflowException(f"final_summary failed: {str(e)}")

default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    'on_failure_callback': alert_on_failure
}

with DAG(
    dag_id='customer_sync',
    schedule='@daily',
    start_date=datetime(2023, 1, 1),
    catchup=False,

```



```

        default_args=default_args,
        on_failure_callback=alert_on_failure
    ) as dag:

        validate_file_task = PythonOperator(
            task_id='validate_file',
            python_callable=validate_file
        )

        create_and_load_task = PythonOperator(
            task_id='create_and_load',
            python_callable=create_and_load
        )

        enrich_customers_task = PythonOperator(
            task_id='enrich_customers',
            python_callable=enrich_customers
        )

        quality_check_task = BranchPythonOperator(
            task_id='quality_check',
            python_callable=quality_check
        )

        all_clear_task = PythonOperator(
            task_id='all_clear',
            python_callable=all_clear
        )

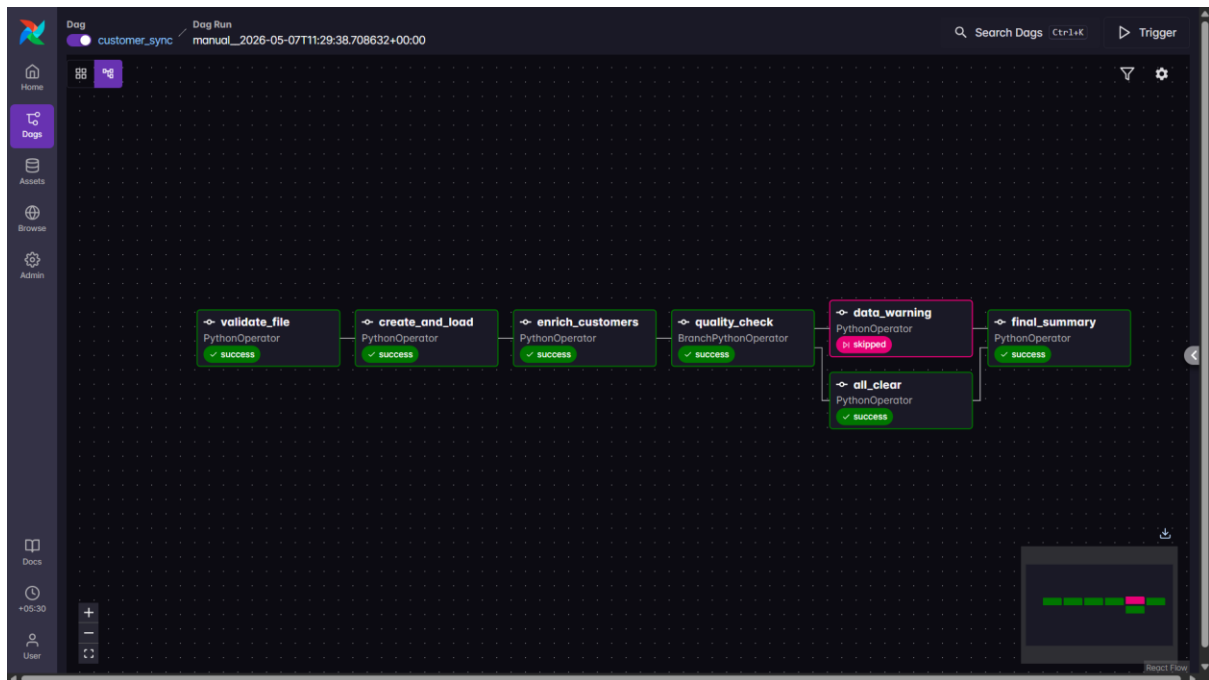
        data_warning_task = PythonOperator(
            task_id='data_warning',
            python_callable=data_warning
        )

        final_summary_task = PythonOperator(
            task_id='final_summary',
            python_callable=final_summary,
            trigger_rule='none_failed_min_one_success'
        )

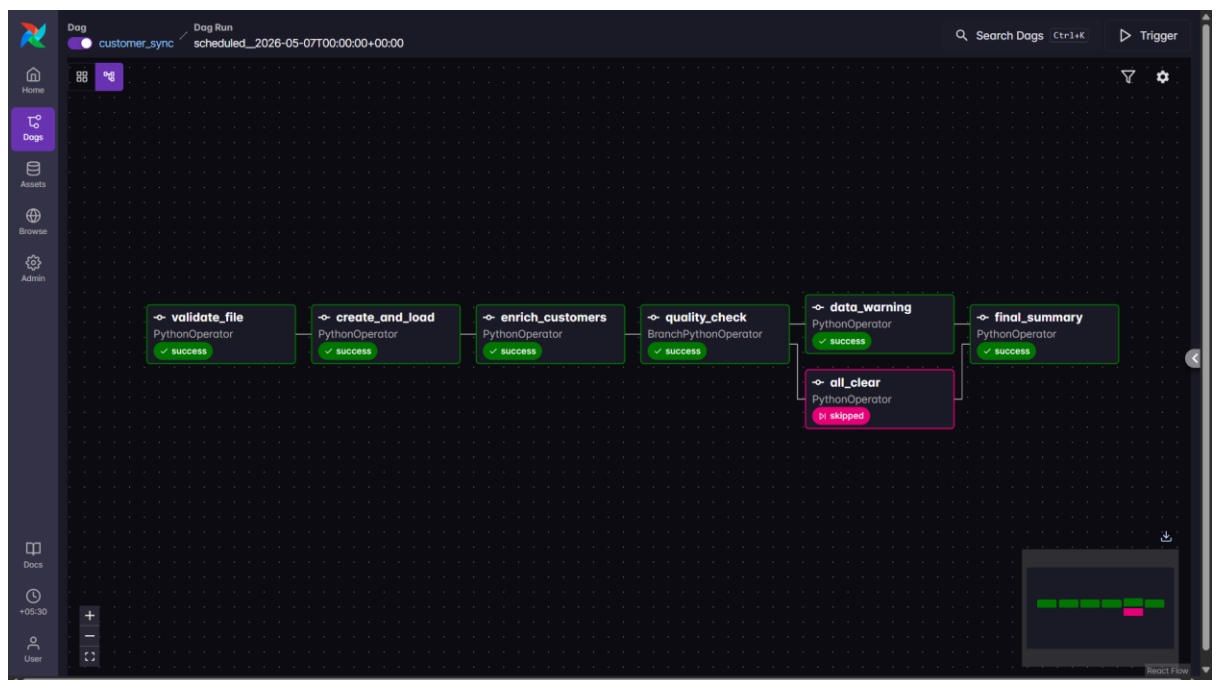
        validate_file_task >> create_and_load_task >> enrich_customers_task >>
quality_check_task
        quality_check_task >> [all_clear_task, data_warning_task] >>
final_summary_task

```

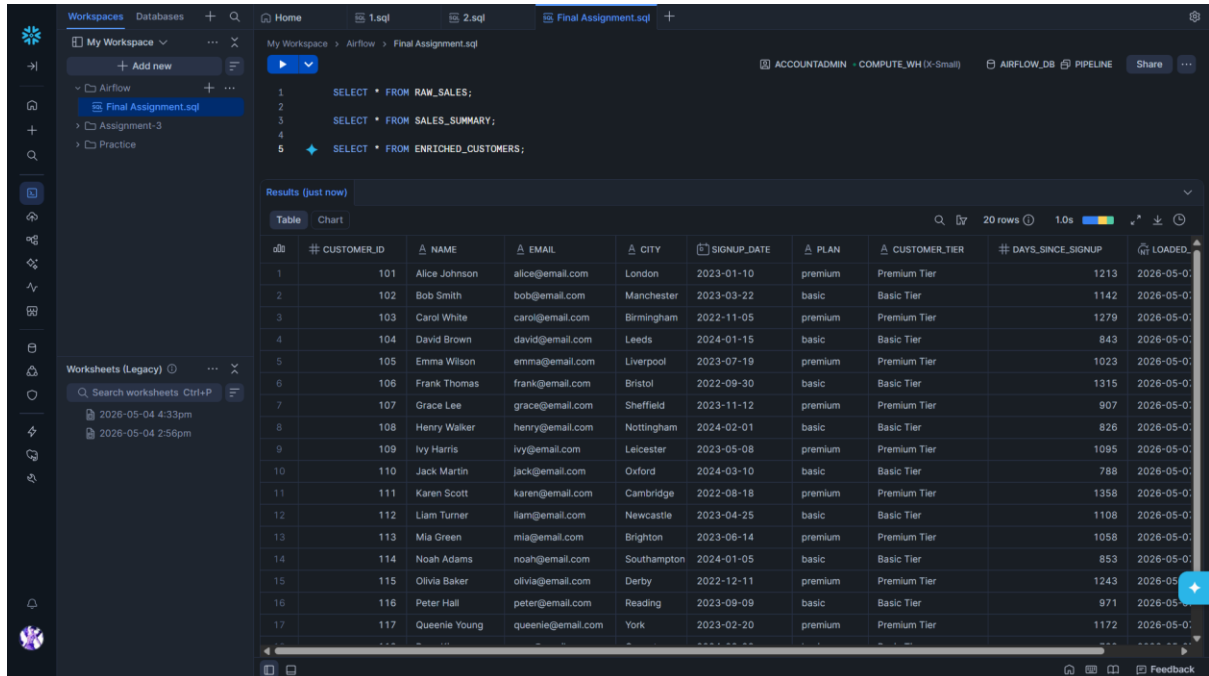
1. Graph View — Run A: all_clear GREEN, data_warning pink (skipped), final_summary GREEN



2. Graph View — Run B (add a bad row to trigger warning): data_warning GREEN, all_clear pink



3. Snowflake Worksheet — SELECT * FROM ENRICHED_CUSTOMERS showing customer_tier and days_since_signup



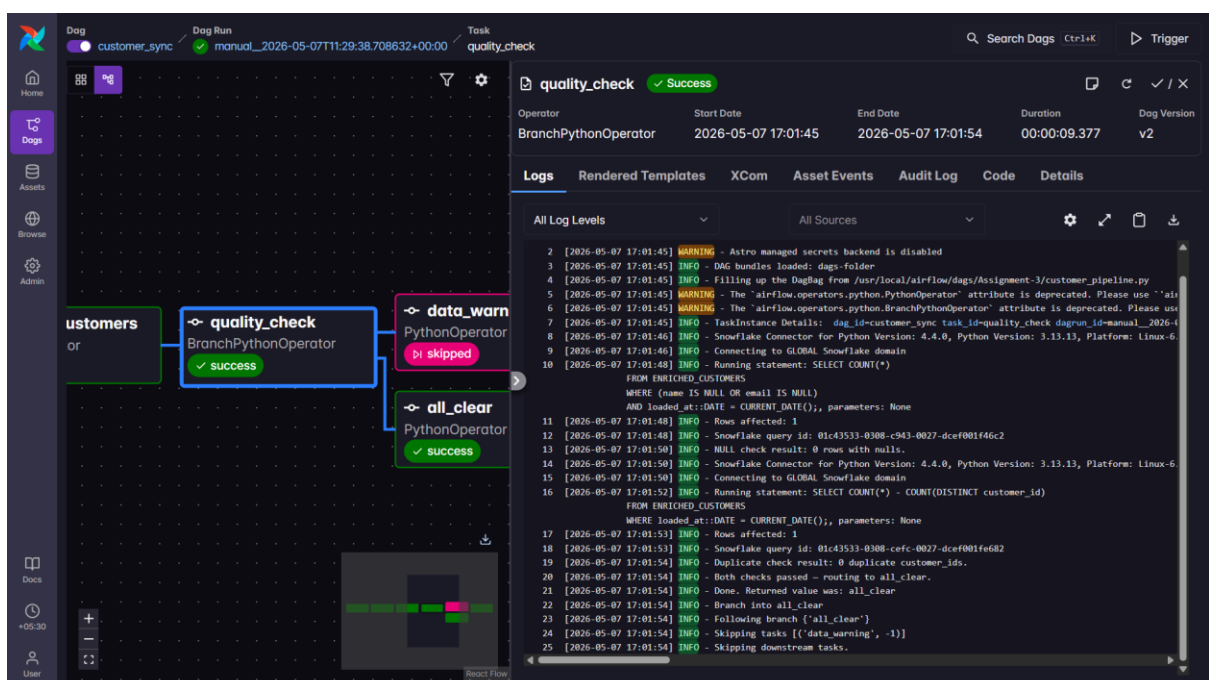
The screenshot shows a Snowflake Worksheet interface. The SQL query is:

```
1 SELECT * FROM RAW_SALES;  
2  
3 SELECT * FROM SALES_SUMMARY;  
4  
5 SELECT * FROM ENRICHED_CUSTOMERS;
```

The results are displayed in a table with the following columns: CUSTOMER_ID, NAME, EMAIL, CITY, SIGNUP_DATE, PLAN, CUSTOMER_TIER, DAYS_SINCE_SIGNUP, and LOADED. The table contains 17 rows of data.

#	CUSTOMER_ID	NAME	EMAIL	CITY	SIGNUP_DATE	PLAN	CUSTOMER_TIER	DAYS_SINCE_SIGNUP	LOADED
1	101	Alice Johnson	alice@email.com	London	2023-01-10	premium	Premium Tier	1213	2026-05-07
2	102	Bob Smith	bob@email.com	Manchester	2023-03-22	basic	Basic Tier	1142	2026-05-07
3	103	Carol White	carol@email.com	Birmingham	2022-11-05	premium	Premium Tier	1279	2026-05-07
4	104	David Brown	david@email.com	Leeds	2024-01-15	basic	Basic Tier	843	2026-05-07
5	105	Emma Wilson	emma@email.com	Liverpool	2023-07-19	premium	Premium Tier	1023	2026-05-07
6	106	Frank Thomas	frank@email.com	Bristol	2022-09-30	basic	Basic Tier	1315	2026-05-07
7	107	Grace Lee	grace@email.com	Sheffield	2023-11-12	premium	Premium Tier	907	2026-05-07
8	108	Henry Walker	henry@email.com	Nottingham	2024-02-01	basic	Basic Tier	826	2026-05-07
9	109	Ivy Harris	ivy@email.com	Leicester	2023-05-08	premium	Premium Tier	1095	2026-05-07
10	110	Jack Martin	jack@email.com	Oxford	2024-03-10	basic	Basic Tier	788	2026-05-07
11	111	Karen Scott	karen@email.com	Cambridge	2022-08-18	premium	Premium Tier	1358	2026-05-07
12	112	Liam Turner	liam@email.com	Newcastle	2023-04-25	basic	Basic Tier	1108	2026-05-07
13	113	Mia Green	mia@email.com	Brighton	2023-06-14	premium	Premium Tier	1058	2026-05-07
14	114	Noah Adams	noah@email.com	Southampton	2024-01-05	basic	Basic Tier	853	2026-05-07
15	115	Olivia Baker	olivia@email.com	Derby	2022-12-11	premium	Premium Tier	1243	2026-05-07
16	116	Peter Hall	peter@email.com	Reading	2023-09-09	basic	Basic Tier	971	2026-05-07
17	117	Queenie Young	queenie@email.com	York	2023-02-20	premium	Premium Tier	1172	2026-05-07

4. Task Log of quality_check — both check values logged before the branch decision



The screenshot shows the Airflow Task Log for the `quality_check` task. The task is a `BranchPythonOperator` and has a status of `Success`. The log shows the following steps:

- `quality_check` (BranchPythonOperator) - `success`
- `data_warn` (PythonOperator) - `skipped`
- `all_clear` (PythonOperator) - `success`

The log also shows the following messages:

- `INFO - Snowflake query id: 01c43533-0308-c943-0027-dcef001f46c2`
- `INFO - NULL check result: 0 rows with nulls.`
- `INFO - Snowflake query id: 01c43533-0308-cefc-0027-dcef001f682`
- `INFO - Duplicate check result: 0 duplicate customer_ids.`
- `INFO - Both checks passed - routing to all_clear.`
- `INFO - Done. Returned value was: all_clear`
- `INFO - Branch into all_clear`
- `INFO - Following branch ('all_clear')`
- `INFO - Skipping task [('data_warn', -1)]`
- `INFO - Skipping downstream tasks.`

5. Task Log of final_summary — tier breakdown report with averages

The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links: Home, Dags, Assets, Browse, and Admin. The main area shows a DAG named 'customer_sync' with a task 'final_summary' highlighted in a blue box. The task status is 'Success'. The task details panel on the right shows the operator 'PythonOperator', start date '2026-05-07 17:33:33', end date '2026-05-07 17:33:38', duration '00:00:04.478', and DAG version 'v2'. The 'Logs' tab is active, displaying a log message source details section with the following log entries:

```
2 [2026-05-07 17:33:33] WARNING - Astro managed secrets backend is disabled
3 [2026-05-07 17:33:33] INFO - DAG bundles loaded: dags-folder
4 [2026-05-07 17:33:33] INFO - Filling up the dagbag from /usr/local/airflow/dags/Assignment-3/customer_pipeline.py
5 [2026-05-07 17:33:34] WARNING - The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators.
6 [2026-05-07 17:33:34] WARNING - The 'airflow.operators.python.BranchPythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.oper
7 [2026-05-07 17:33:34] INFO - TaskInstance Details: dag_id=customer_sync task_id=final_summary dagrun_id=manual_2026-05-07T12:02:48.160801+00:00 map_ind
8 [2026-05-07 17:33:34] INFO - Snowflake Connector for Python Version: 4.4.0, Python Version: 3.13.13, Platform: Linux-6.6.87.2-microsoft-standard-WSL2-x86
9 [2026-05-07 17:33:34] INFO - Connecting to GLOBAL Snowflake domain
10 [2026-05-07 17:33:36] INFO - Running statement: SELECT
    COUNT(*) AS total_customers,
    SUM(CASE WHEN customer_tier = 'Premium Tier' THEN 1 ELSE 0 END) AS premium_count,
    SUM(CASE WHEN customer_tier = 'Basic Tier' THEN 1 ELSE 0 END) AS basic_count,
    MAX(DAYS_SINCE_SIGNUP) AS max_days_since_signup,
    MODE(city) AS top_city
FROM ENRICHED_CUSTOMERS
WHERE loaded_at::DATE = CURRENT_DATE();; parameters: None
11 [2026-05-07 17:33:37] INFO - Rows affected: 1
12 [2026-05-07 17:33:37] INFO - Snowflake query id: 81c3553-4308-c943-8927-dcef001f4712
13 [2026-05-07 17:33:38] INFO - Total Customers: 20
14 [2026-05-07 17:33:38] INFO - Premium Tier: 10
15 [2026-05-07 17:33:38] INFO - Basic Tier: 10
16 [2026-05-07 17:33:38] INFO - Average Days Since Signup: 1036.9
17 [2026-05-07 17:33:38] INFO - City with Most Customers: Nottingham
18 [2026-05-07 17:33:38] INFO - Done. Returned value was: None
```